

Folder Structure

```
/01-ai-tools
  /02052510-Blerta-Hoxha
    research.md
    reflection.md
    references.md
```

research.md

AI Coding and Development Tools

Student Name: Blerta Hoxha

Student ID: 02052510

Topic: Topic 0 — AI Coding and Development Tools

Introduction

Artificial Intelligence has changed the way software engineers build applications. Modern developers no longer work only by writing code manually. Today, many programmers use AI-powered tools that can suggest code, detect mistakes, generate documentation, and automate repetitive tasks. These tools help developers work faster and improve productivity.

AI coding tools are becoming popular because software systems are becoming more complex while companies continue demanding faster development. AI does not completely replace software engineers, but it supports them during development.

This research explains different AI coding and development tools, how developers use them, their advantages and disadvantages, and whether these technologies assist or replace programmers.

1. AI Coding Assistants

AI coding assistants are tools that help programmers write code faster. They use machine learning models trained on large amounts of source code. These systems analyze what the programmer is typing and generate suggestions.

One of the most popular tools is GitHub Copilot. It works inside code editors and suggests entire lines or functions while developers write code. It can generate code in different programming languages such as Python, Java, JavaScript, and C++.

Another important assistant is ChatGPT by OpenAI. Developers use ChatGPT to explain programming concepts, debug errors, generate algorithms, and understand documentation.

Benefits of AI Coding Assistants

- Faster code writing
- Reduced repetitive tasks
- Help for beginners learning programming
- Faster debugging support
- Improved productivity

Limitations of AI Coding Assistants

- AI-generated code may contain bugs
- Security vulnerabilities can appear in generated code
- Developers may become overdependent on AI
- AI sometimes produces incorrect or outdated solutions
- Generated code still requires human review

Although these tools are powerful, developers still need programming knowledge to verify whether the generated code is correct.

2. AI Code Completion Tools

Code completion tools predict what developers want to write next. Traditional autocomplete systems only suggested keywords, but AI-powered completion tools understand context.

Examples include:

- GitHub Copilot
- Amazon CodeWhisperer
- Tabnine
- JetBrains AI Assistant

These systems analyze the surrounding code and provide intelligent recommendations.

For example, when a developer writes the beginning of a function, the AI may generate the entire implementation automatically.

How Developers Use Them

Developers use these tools to:

- Write repetitive code faster
- Generate boilerplate code
- Create API calls
- Produce test cases
- Learn syntax quickly

Problems and Risks

Sometimes AI completion systems generate code that looks correct but contains hidden logical errors. Junior developers may accept suggestions without fully understanding them.

There are also copyright concerns because some AI models are trained on public repositories.

3. AI Documentation Generators

Writing documentation is an important but time-consuming activity in software engineering. AI documentation tools help developers automatically generate comments, summaries, and explanations.

Examples include:

- Mintlify
- GitBook AI
- Swimm
- ChatGPT

These tools can:

- Explain functions automatically
- Generate README files
- Create API documentation
- Summarize complex code

Advantages

- Saves time
- Improves project organization
- Helps teams understand code faster
- Reduces missing documentation

Limitations

Automatically generated documentation may sometimes be incomplete or inaccurate. Developers must still review the content before publishing it.

4. AI Project Assistants and DevOps Tools

AI is also used in project management and DevOps.

DevOps combines software development and deployment operations. AI tools help automate monitoring, deployment, testing, and issue detection.

Examples include:

- GitHub Actions
- Jenkins AI plugins
- Datadog AI monitoring
- Azure DevOps AI tools

These systems help developers:

- Detect deployment failures
- Monitor application performance
- Predict system problems
- Automate workflows

Benefits

- Faster deployment
- Reduced human error
- Better system monitoring
- Improved reliability

Challenges

AI automation systems are not always perfect. Incorrect automated actions may cause production issues if engineers do not supervise the process.

5. Do AI Tools Replace Developers?

Many people ask whether AI will replace software engineers completely.

Currently, AI tools mainly assist developers instead of replacing them. AI can generate code quickly, but software engineering involves much more than coding.

Software engineers still need:

- Problem-solving skills
- Communication skills
- System design knowledge
- Critical thinking
- Security understanding
- Ethical decision-making
- Team collaboration

AI tools cannot fully understand business requirements, user needs, or long-term project goals the same way humans can.

Senior developers are especially important because they review architecture decisions and verify whether AI-generated solutions are safe and maintainable.

Instead of replacement, the industry is experiencing transformation. Developers who learn how to work effectively with AI will likely become more productive.

6. Future of AI in Software Engineering

AI will continue improving in the future. More companies are integrating AI into development workflows.

Future software engineers may spend less time writing repetitive code and more time focusing on:

- software architecture,
- product design,
- system integration,
- AI supervision,
- and quality verification.

AI may also increase the importance of human creativity and communication.

Engineers will need continuous learning because technologies evolve rapidly.

Conclusion

AI coding and development tools are changing software engineering significantly. Tools like GitHub Copilot, ChatGPT, and AI DevOps systems improve productivity and automate repetitive tasks.

However, AI still has limitations. Generated code may contain bugs, security problems, or incorrect logic. Human supervision remains extremely important.

The future of software engineering will likely involve collaboration between humans and AI systems. Developers who combine technical knowledge with critical thinking and adaptability will remain valuable in the AI era.

reflection.md

Personal Reflection

During this research, I learned that Artificial Intelligence is becoming an important part of software engineering. Before researching this topic, I mostly thought AI tools were only simple assistants for writing code faster. After reading articles and industry discussions, I understood that AI affects many parts of software development including testing, documentation, deployment, and project management.

The most surprising thing for me was how quickly companies are adopting AI tools such as GitHub Copilot and ChatGPT. Many developers already use these tools daily to improve productivity.

At the same time, I also discovered several concerns. One major concern is that developers may become too dependent on AI-generated code without fully understanding it. This could create security vulnerabilities and poor software quality. I think engineers must continue learning programming fundamentals instead of relying completely on automation.

I believe AI creates many opportunities for future software engineers. Developers can save time on repetitive tasks and focus more on creative problem solving, architecture design, and innovation. AI may also help beginners learn faster.

In my opinion, human skills will remain extremely important in the future. Communication, teamwork, critical thinking, and decision-making cannot easily be replaced by AI systems. Software engineering is not only about writing code; it also involves understanding people, solving business problems, and collaborating in teams.

This research changed my opinion about the future of programming jobs. I no longer think AI will simply replace programmers. Instead, I think the role of software engineers will evolve. Developers who learn how to work together with AI tools will have better opportunities.

Personally, I plan to adapt by continuously learning new technologies while also improving my problem-solving and communication skills. I believe future engineers must balance technical knowledge with adaptability and critical thinking.

references.md

References

1. GitHub Copilot Official Website
<https://github.com/features/copilot>
2. OpenAI ChatGPT
<https://openai.com/chatgpt>
3. Amazon CodeWhisperer Documentation
<https://aws.amazon.com/codewhisperer/>
4. JetBrains AI Assistant
<https://www.jetbrains.com/ai/>
5. McKinsey & Company — The State of AI Report
<https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai>
6. Stack Overflow Developer Survey
<https://survey.stackoverflow.co/>

7. GitHub Research on AI and Developers
<https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/>
 8. Microsoft AI in Software Development Research
<https://www.microsoft.com/en-us/research/>
 9. Tabnine AI Coding Assistant
<https://www.tabnine.com/>
 10. Datadog AI Monitoring Tools
<https://www.datadoghq.com/>
-

Suggested Git Commit Messages

```
git commit -m "Created assignment folder structure"
```

```
git commit -m "Added research about AI coding tools"
```

```
git commit -m "Added personal reflection and references"
```